Report

SBA 927 — Business Text Analytics: Natural Language Processing Techniques

Amira Lions | Final report | October 30, 2026

Overview

This assessment applied natural language processing (NLP) techniques to 12 text entries from the supplied SBA927.txt dataset, drawn from dialogue and narrative descriptions associated with iRobot. Work included text preprocessing, stemming and lemmatization, named entity recognition (NER), part-of-speech (POS) tagging, sentiment prompt design, comparative sentiment analysis, and a pre-trained transformer experiment. The purpose was to assess how these techniques could support interpretation of unstructured business text while recognizing their limitations.

The dataset is film-related rather than actual customer feedback. Accordingly, conclusions about business applications are illustrative and should not be treated as measured customer-satisfaction findings.

Steps

1. Text loading, tokenization, and preprocessing

Loaded 12 entries from SBA927.txt and tokenized text with NLTK. Preprocessing included lowercasing, punctuation removal, and filtering stop words, while deliberately retaining negations such as “not,” “never,” “no,” and “n’t” because removing them can reverse sentiment meaning. A test using “I don’t like this product. It is not good.” retained [“n’t”, “like”, “product”, “not”, “good”]. The token list shows that preprocessing was functional, although preserving “n’t” separately also demonstrates how tokenization can affect downstream interpretation.

2. Stemming and lemmatization

Applied NLTK PorterStemmer and WordNetLemmatizer to processed text. Stemming reduces words to heuristic stems that need not be valid dictionary words; in observed output, “quite” became “quit,” illustrating possible meaning loss. Lemmatization is more conservative and generally seeks valid lexical forms, but its quality depends on word context and part-of-speech information. For business reporting, lemmatization is often preferable when human-readable terms matter; stemming can be useful for broad keyword grouping.

3. Named entity recognition (NER)

Used NLTK named-entity chunking to identify names and entity categories. Execution succeeded after resolving a missing NumPy dependency. The initial traceback identified the call from the project script but located the actual ModuleNotFoundError within NLTK’s maxent.py; installing NumPy with uv resolved the runtime issue. This illustrates why traceback analysis should distinguish the application call site from the dependency where an exception occurs.

NER output contained semantic errors: “Sonny” was labeled GPE in one instance, and “Robot” and “Program” were labeled PERSON. Therefore, a successful run is not evidence of correct entity classification. In business documents, incorrect identification of customers, locations, and organizations could contaminate downstream analysis and should be reviewed.

4. Part-of-speech tagging

Applied NLTK POS tagging to identify grammatical roles. Examples of observed limitations included “Detective” tagged JJ (adjective) and “Spooner” tagged NN (common noun) rather than NNP (proper noun). POS tags can help distinguish actions, attributes, and named references, but ambiguous context, titles, and unfamiliar names require careful interpretation.

5. Sentiment-analysis prompt design

Created a reusable Python prompt-generation function, create_sentiment_prompt(text), asking a language model to classify each entry as Positive, Negative, or Neutral, consider negation and context, and respond with a sentiment label and explanation. The prompts were constructed in Python; they were not submitted through a model API. The ChatGPT comparison classifications were obtained through conversational analysis, not a programmatic API integration.

Prompt structure (condensed): Analyze the sentiment of the supplied text; choose Positive, Negative, or Neutral; account for context and negation; return “Sentiment:” and “Explanation:”.

6. Sentiment analysis and evaluation

Ran NLTK VADER locally against all 12 original entries, recording compound scores to four decimal places, and compared the results with GPT-6 conversational classifications. The two approaches agreed on 5 of 12 entries (41.7%) and disagreed on 7. This is agreement, NOT accuracy: no independently labeled gold-standard dataset was available. Sarcasm, mixed tone, descriptive prose, and negation contributed to differences.

7. Pre-trained language-model experiment

Installed Hugging Face Transformers 5.22.1 and PyTorch 2.15.0+cpu under Python 3.14.7, then loaded the publicly available pre-trained distilbert/distilbert-base-uncased-finetuned-sst-2-english model. The first-entry test and subsequent inference on all 12 entries completed successfully on the CPU. The model predicted 9 Positive and 3 Negative labels. Its SST-2 classifier is binary, so it cannot directly return Neutral; comparisons with three-class methods must be qualified.

The model was used for inference only. No fine-tuning or additional training was performed. The model was downloaded without a Hugging Face token; Windows cache-symlink warnings did not prevent execution.

Comparison of sentiment results

Text	VADER	ChatGPT GPT-6	DistilBERT	DistilBERT confidence
1	Neutral	Negative	Negative	.9998
2	Negative	Negative	Positive	.9687
3	Positive	Neutral	Positive	.9799
4	Neutral	Neutral	Positive	.9973
5	Positive	Negative	Positive	.9098
6	Positive	Positive	Positive	.9997
7	Neutral	Neutral	Negative	.9996
8	Negative	Negative	Positive	.8626
9	Positive	Neutral	Positive	.9986
10	Positive	Negative	Positive	.9813
11	Positive	Neutral	Positive	.9983
12	Positive	Neutral	Negative	.9907

Selected findings: Text 1 (sarcastic “I told you so”) was Neutral under VADER but Negative under ChatGPT and DistilBERT (0.9998). Text 2, which mentions a potential threat to humanity, was Negative under VADER and ChatGPT but Positive under DistilBERT (0.9687). Text 8 was Negative under VADER and ChatGPT but Positive under DistilBERT (0.8626). Text 10 (“I did not murder him!”) was Positive under VADER and DistilBERT but Negative under ChatGPT. These divergences emphasize that a model’s confidence score is not proof of correctness.

Business interpretation and model limitations

Tokenization and normalization help prepare customer comments for trend analysis; NER can surface organizations, products, and places; POS tagging can identify descriptive language; and sentiment classification can help prioritize feedback. However, errors can arise from sarcasm, context, negation, domain mismatch, and ambiguous entity names. The tested film-related dataset does not establish performance on

real customer reviews. For consequential business decisions, a domain-matched, human-labeled evaluation set, error review, and ongoing monitoring are necessary.

No quantitative accuracy, precision, recall, or fairness/bias benchmark can be established from these results alone. Bias can arise from training-data domains and language patterns; in particular, the SST-2 binary label design forces neutral material into Positive or Negative. Model selection and human review are therefore part of responsible deployment.

Conclusion

All seven numbered requirements were executed and documented. The work demonstrated reproducible NLP preprocessing and analysis, prompt construction, a local lexicon-based sentiment baseline, and inference with a pre-trained transformer model. The most important finding was not that one method "won," but that plausible NLP systems can disagree substantially on the same text. The 41.7% VADER–ChatGPT agreement and DistilBERT's confidently divergent predictions support cautious interpretation, domain-specific testing, and human validation before using NLP outputs to guide business strategy.

Appendix

Project artifacts

- SBA927.txt — supplied 12-entry source dataset
- sba_927.py — main NLP processing and analysis script
- pretrained_model_test.py — reproducible transformer inference experiment
- sentiment_comparison.csv — VADER versus ChatGPT results
- sentiment_analysis_notes.md — Requirement 6 methodology and analysis
- pretrained_model_results.csv — DistilBERT labels and confidence
- pretrained_model_notes.md — Requirement 7 methodology and observations
- pyproject.toml and uv.lock — Python dependencies and reproducibility
- README.md — project documentation

Reproduction and evidence

From the project root, uv run python sba_927.py runs the primary script and uv run python pretrained_model_test.py runs the transformer experiment. The transformer model may need to download weights on first execution. The CSV and Markdown files provide preserved observations. Terminal outputs were checked during development; screenshots can be appended if specifically required by the instructor.